1.4、鼠标事件

[OpenCV: Mouse as a Paint-Brush]

- 这part看懂官网的两个代码例子就差不多了

Code1

```
import numpy as np
import cv2 as cv

# 這是滑鼠的回呼函數（callback），格式固定要這樣寫
def draw_circle(event, x, y, flags, param):
    if event == cv.EVENT_LBUTTONDBLCLK:
        cv.circle(img, (x, y), 100, (255, 0, 0), -1)   # -1 = 填滿

# 建立一張黑色圖 + 視窗 + 綁定滑鼠事件
img = np.zeros((512, 512, 3), np.uint8)
cv.namedWindow('image')
cv.setMouseCallback('image', draw_circle)

while True:
    cv.imshow('image', img)
    if cv.waitKey(20) & 0xFF == 27:    # 按 ESC 結束
        break

cv.destroyAllWindows()
```

- 这个程序代码是 OpenCV 最经典的入门示例之一："在图片上双击左键 → 在点击位置画一个蓝色实心圆"。它展示了 OpenCV 如何通过**鼠标回调函数（callback）**来实现简单的鼠标交互

1. 定义鼠标回调函数（最核心的部分）

```
def draw_circle(event, x, y, flags, param):
    if event == cv.EVENT_LBUTTONDBLCLK:
        cv.circle(img, (x, y), 100, (255, 0, 0), -1)   # -1 = 填充
```

这个函数名字可以随便取，但参数必须是固定的 5 个，按这个顺序：

参数	含义	本例中怎么用
event	发生了什么鼠标事件（整数代码）	只关心左键双击（EVENT_LBUTTONDBLCLK）
x	鼠标当前 x 坐标（水平方向）	作为圆心的 x 位置
y	鼠标当前 y 坐标（垂直方向）	作为圆心的 y 位置
flags	附加状态标志（是否按 Shift/Ctrl 等）	本例完全没用到，可以忽略
param	用户自定义的额外参数（可选传递）	本例也没用到（直接用全局变量 img）

常见鼠标事件：

- cv.EVENT_LBUTTONDOWN → 左鍵按下
- cv.EVENT_LBUTTONUP → 左鍵放開
- cv.EVENT_MOUSEMOVE → 滑鼠移動
- cv.EVENT_LBUTTONDBLCLK → 左鍵雙擊
- cv.EVENT_RBUTTONDOWN → 右鍵按下
- cv.EVENT_MBUTTONDOWN → 中鍵按下

2. 创建图像、窗口、绑定鼠标事件

```
img = np.zeros((512, 512, 3), np.uint8)
cv.namedWindow('image')
```

- 创建一个名叫 'image' 的窗口。
- 这个窗口名字很重要，后面的 imshow 和 setMouseCallback 都要用同一个名字。

```
cv.setMouseCallback('image', draw_circle)
```

- 关键绑定：告诉 OpenCV，“只要在名叫 'image' 的窗口发生鼠标事件，就调用 draw_circle 这个函数”。
- 第三个参数（可选的用户数据）没传，所以 param 会是 None。

3. 事件循环（让窗口保持打开并持续刷新）

```
while True:
    cv.imshow('image', img)
```

```
    if cv.waitKey(20) & 0xFF == 27:    # 按 ESC 结束
        break
```

这是 OpenCV GUI 程序的标准无限循环结构：（这块循环要是不理解就类比单片机的逻辑，作用仅仅是不停地显示img和读按键，我理解的是callback也相当于是一个死循环在等待鼠标事件）

1. cv.imshow('image', img)
   → 每次循环都把当前的 img 显示到窗口上。
   （因为我们在回调函数里直接修改 img，所以画的圆会立刻显示出来）
2. cv.waitKey(20)
   → 等待 20 毫秒，检查有没有键盘输入。
   → 返回按下的键的 ASCII 码（没按键就返回 -1）。
3. & 0xFF
   → 为了兼容性，只取低 8 位（有些系统可能返回负值）。
4. == 27
   → 27 是 ESC 键的 ASCII 码。
   → 按下 ESC 就跳出 while 循环，结束程序。

整体逻辑流程（思路总结）

1. 导入必要的库
2. 定义回调函数 draw_circle
   └─ 只在左键双击时 → 在鼠标位置 (x,y) 画一个蓝色实心圆到 img 上
3. 创建一张黑色图像 img
4. 创建窗口 'image'
5. 把 draw_circle 绑定到 'image' 窗口的鼠标事件
6. 进入无限循环：
   ├─ 显示当前 img 到窗口
   ├─ 每 20 毫秒检查一次键盘输入
   └─ 如果按 ESC → 跳出循环
7. 关闭所有窗口，程序结束

Code2

```
import numpy as np
import cv2 as cv

drawing = False       # 是否正在拖曳
mode = True           # True=畫矩形，False=畫圓
ix, iy = -1, -1       # 起點座標
img = np.zeros((512, 512, 3), np.uint8)
```

```python
img_copy = img.copy()   # 用來保存「沒拖曳時」的圖，防止閃爍

def draw_shape(event, x, y, flags, param):
    global ix, iy, drawing, mode, img, img_copy

    if event == cv.EVENT_LBUTTONDOWN:
        drawing = True
        ix, iy = x, y
        img_copy = img.copy()   # 記錄按下時的原始圖

    elif event == cv.EVENT_MOUSEMOVE:
        if drawing:
            img = img_copy.copy()  # 每次都從原始圖開始，避免重疊殘影
            if mode:  # 矩形模式
                cv.rectangle(img, (ix, iy), (x, y), (0, 255, 0), 2)   # 2=框線
粗細
            else:     # 自由畫圓點（像筆刷）
                cv.circle(img, (x, y), 5, (0, 0, 255), -1)

    elif event == cv.EVENT_LBUTTONUP:
        drawing = False
        if mode:
            cv.rectangle(img, (ix, iy), (x, y), (0, 255, 0), 2)
        else:
            cv.circle(img, (x, y), 5, (0, 0, 255), -1)

cv.namedWindow('image')
cv.setMouseCallback('image', draw_shape)

while True:
    cv.imshow('image', img)
    key = cv.waitKey(1) & 0xFF

    if key == ord('m'):        # 按 m 切換模式
        mode = not mode
        print("現在模式: ", "矩形" if mode else "圓點筆刷")

    elif key == 27:            # ESC 結束
        break

cv.destroyAllWindows()
```

- 有几个巧思来实现拖拽画图和其显示：
  - 用全局变量 drawing、mode、ix,iy 记录“用户当前在做什么”，因为要修改全局变量，所以用 global 声明

- 用一个变量drawing来控制
- 把当前图像完整备份到 img_copy（后面拖拽预览都从按下左键这里开始），拖拽过程中img = img_copy.copy()，→ 每次鼠标移动都把图像“复原”到拖拽开始前的状态，然后再画新的形状 → 实现“实时预览但不留残影”的效果